

LAB GUIDE WITH SOLUTIONS



Module 3: Advanced SQL Queries

Module 3 - 8 Hours

Advanced SQL Queries

How to use this lab guide

Eight labs for a four-hour session. Solutions and real expected output are inline.

Lab	Title	Time
3.1	Inner joins across the bank	30 min
3.2	Outer joins and the silent bug	30 min
3.3	Self joins and non-equi joins	30 min
3.4	Aggregates, GROUP BY and HAVING	35 min
3.5	Pivoting with conditional aggregation	25 min
3.6	Subqueries	35 min
3.7	Set operators and reconciliation	20 min
3.8	Assemble the branch performance pack	25 min

Start position: `icicibank` with 8 branches, 30 customers, 36 accounts, 495 transactions, 12 loans, 24 employees and 20 fixed deposits. Verify before you start. Every expected output below comes from that exact dataset.

Lab 3.1 - Inner joins across the bank

Step 1 - See the accident first

```
SELECT COUNT(*) FROM customer c, account a;
```

```
count
-----
1080
```

Thirty customers times thirty-six accounts. No join condition, so every customer was paired with every account. Remember what this looks like, because at production volume the query never finishes.

Step 2 - Add the join condition

```
SELECT c.customer_id,
       c.first_name || ' ' || c.last_name AS customer_name,
       a.account_number, a.account_type, a.balance
FROM   customer c
       JOIN account a ON a.customer_id = c.customer_id
ORDER BY c.customer_id
LIMIT 8;
```

customer_id	customer_name	account_number	account_type	balance
1001	Aarav Sharma	ICIC0000009001	CURRENT	35859.46
1002	Diya Nair	ICIC0000009002	SALARY	43522.54
1003	Rohan Iyer	ICIC0000009003	SAVINGS	341244.80
1004	Meera Reddy	ICIC0000009004	SAVINGS	421425.91
1005	Karthik Gupta	ICIC0000009005	CURRENT	91729.06
1005	Karthik Gupta	ICIC0000009006	SALARY	27637.82
1006	Ananya Menon	ICIC0000009007	SALARY	390474.85
1007	Vikram Patel	ICIC0000009008	SAVINGS	74946.54

Note customer 1005 appearing twice. They hold two accounts, so the join produces two rows. **That is not a bug** - but it is exactly the fan-out that will break your counts in Lab 3.4.

Step 3 - Prove the ambiguity error

```
SELECT customer_id FROM customer JOIN account ON account.customer_id = customer.customer_id;
```

```
ERROR: column reference "customer_id" is ambiguous
```

Both tables have a column of that name. Alias the tables and qualify the column.

Step 4 - Three tables

Everyone banking at Fort Mumbai Main, richest first.

```

SELECT b.branch_name,
       c.first_name || ' ' || c.last_name AS customer_name,
       a.account_number, a.balance
FROM   branch b
       JOIN customer c ON c.home_branch_id = b.branch_id
       JOIN account a ON a.customer_id    = c.customer_id
WHERE  b.branch_id = 1
ORDER BY a.balance DESC;

```

branch_name	customer_name	account_number	balance
Fort Mumbai Main	Joseph Shetty	ICIC0000009020	408783.99
Fort Mumbai Main	Farhan Gupta	ICIC0000009029	274406.29
Fort Mumbai Main	Farhan Gupta	ICIC0000009030	163114.34
Fort Mumbai Main	Arjun Singh	ICIC0000009010	49313.33
Fort Mumbai Main	Aarav Sharma	ICIC0000009001	35859.46

Five rows from four customers, because Farhan Gupta holds two accounts.

Step 5 - Four tables

```

SELECT b.branch_name,
       c.first_name || ' ' || c.last_name AS customer_name,
       t.txn_date, t.txn_type, t.amount, t.channel
FROM   branch      b
       JOIN customer c ON c.home_branch_id = b.branch_id
       JOIN account  a ON a.customer_id    = c.customer_id
       JOIN account_txn t ON t.account_id   = a.account_id
WHERE  t.amount >= 200000
      AND t.status = 'SUCCESS'
ORDER BY t.amount DESC
LIMIT 10;

```

Four tables, three `ON` conditions. `account` supplies no output columns; it exists only to connect customers to transactions. That is normal.

Step 6 - Count the join conditions

```

-- Deliberately drop one condition
SELECT COUNT(*)
FROM   branch b
       JOIN customer c ON c.home_branch_id = b.branch_id
       JOIN account a ON a.customer_id    = c.customer_id
       JOIN account_txn t ON 1 = 1;

```

The row count explodes: every branch-customer-account combination is paired with every one of the 495 transactions. **`n` tables need `n-1` join conditions.**

Lab 3.2 - Outer joins and the silent bug

Step 1 - Inner versus left

```
SELECT COUNT(*) AS inner_rows
FROM   customer c JOIN fixed_deposit f ON f.customer_id = c.customer_id;
-- 20

SELECT COUNT(*) AS left_rows
FROM   customer c LEFT JOIN fixed_deposit f ON f.customer_id = c.customer_id;
-- 30
```

Twenty customers hold a Smart Saver deposit. The `LEFT JOIN` keeps the other ten as well, with NULLs in the deposit columns.

Step 2 - See the NULLs

```
SELECT c.customer_id,
       c.first_name || ' ' || c.last_name AS customer_name,
       f.receipt_number,
       f.principal_amount
FROM   customer c
       LEFT JOIN fixed_deposit f ON f.customer_id = c.customer_id
ORDER  BY c.customer_id
LIMIT 12;
```

Customers 1009 and 1011 show blank receipt and principal. Those blanks are NULL, not zero.

Step 3 - The bug

```
-- Intended: every customer, with their ACTIVE deposit if they have one
SELECT COUNT(*)
FROM   customer c
       LEFT JOIN fixed_deposit f ON f.customer_id = c.customer_id
WHERE  f.status = 'ACTIVE';
-- 18  <-- the ten customers without deposits have vanished

-- Correct
SELECT COUNT(*)
FROM   customer c
       LEFT JOIN fixed_deposit f ON f.customer_id = c.customer_id
                                AND f.status = 'ACTIVE';
-- 30  <-- every customer survives
```

For a customer with no deposit, `f.status` is NULL, so `f.status = 'ACTIVE'` is NULL, and `WHERE` drops the row. The outer join did its job and the `WHERE` clause undid it.

The rule: on an outer join, a condition on the **optional** table goes in `ON`. A condition on the **required** table goes in `WHERE`.

Step 4 - The anti-join

```
-- Accounts that have never seen a transaction
SELECT a.account_id, a.account_number, a.status
FROM   account a
      LEFT JOIN account_txn t ON t.account_id = a.account_id
WHERE  t.txn_id IS NULL;
```

account_id	account_number	status
9001	ICIC0000009001	DORMANT

```
-- Customers with no Smart Saver deposit: the cross-sell list
SELECT c.customer_id, c.first_name, c.last_name
FROM   customer c
      LEFT JOIN fixed_deposit f ON f.customer_id = c.customer_id
WHERE  f.deposit_id IS NULL
ORDER BY c.customer_id;
```

customer_id	first_name	last_name
1009	Arjun	Singh
1011	Rahul	Khan
1013	Sanjay	Joseph
1016	Fatima	Kulkarni
1017	Joseph	Shetty
1019	Manish	Chopra
1022	Swati	Nair
1024	Lakshmi	Reddy
1027	Gaurav	Patel
1029	Suresh	Singh

Ten customers, and marketing now has a call list. **Test the primary key for NULL**, `f.deposit_id`, never a nullable column such as `f.nominee_name`.

Step 5 - Why not RIGHT JOIN

```
SELECT COUNT(*) FROM fixed_deposit f RIGHT JOIN customer c ON f.customer_id = c.customer_id;
-- 30, identical to the LEFT JOIN in step 1
```

Same answer, harder to read. Pick a direction and stay with it.

Lab 3.3 - Self joins and non-equi joins

Step 1 - The ICICI Bank hierarchy

```
SELECT e.employee_id,
       e.first_name || ' ' || e.last_name AS employee,
       e.designation,
       COALESCE(m.first_name || ' ' || m.last_name, '(no manager)') AS reports_to
FROM   employee e
       LEFT JOIN employee m ON m.employee_id = e.manager_id
WHERE  e.branch_id = 1
ORDER  BY e.employee_id;
```

employee_id	employee	designation	reports_to
501	Meera Menon	Branch Manager	(no manager)
502	Sneha Nair	Relationship Manager	Meera Menon
503	Arjun Gupta	Teller	Meera Menon

Step 2 - Prove why LEFT matters

```
SELECT COUNT(*) FROM employee e JOIN      employee m ON m.employee_id = e.manager_id; -- 16
SELECT COUNT(*) FROM employee e LEFT JOIN employee m ON m.employee_id = e.manager_id; -- 24
```

The eight Branch Managers have a NULL `manager_id` and are dropped by the inner join. In an org chart, losing every manager is not a small error.

Step 3 - Managers and their team size

```
SELECT m.employee_id,
       m.first_name || ' ' || m.last_name AS manager,
       m.branch_id,
       COUNT(e.employee_id) AS team_size
FROM   employee m
       LEFT JOIN employee e ON e.manager_id = m.employee_id
WHERE  m.designation = 'Branch Manager'
GROUP  BY m.employee_id, m.first_name, m.last_name, m.branch_id
ORDER  BY m.branch_id;
```

Each Branch Manager has a team of 2.

Step 4 - Build a non-equi join

Risk wants every transaction banded by size.

```
CREATE TABLE txn_band (
  band_code  VARCHAR(10) NOT NULL,
  band_name  VARCHAR(30) NOT NULL,
  min_amount NUMERIC(15,2) NOT NULL,
  max_amount NUMERIC(15,2) NOT NULL,
```

```
CONSTRAINT pk_txn_band PRIMARY KEY (band_code)
);

INSERT INTO txn_band (band_code, band_name, min_amount, max_amount) VALUES
('B1', 'Micro',      0.00,      1000.00),
('B2', 'Small',     1000.00,    10000.00),
('B3', 'Medium',    10000.00,   100000.00),
('B4', 'Large',     100000.00,  200000.00),
('B5', 'Very Large', 200000.00,  9999999.00);
```

```
SELECT b.band_name,
       COUNT(*)           AS txns,
       ROUND(SUM(t.amount), 2) AS total_value
FROM   account_txn t
       JOIN txn_band b ON t.amount >= b.min_amount
                        AND t.amount < b.max_amount
WHERE  t.status = 'SUCCESS'
GROUP BY b.band_code, b.band_name
ORDER BY b.band_code;
```

The join uses `>=` and `<`, not `=`. Note the half-open ranges: a transaction of exactly 10,000 falls in Small, never in both bands. **Overlapping bands would duplicate every boundary transaction**, and the report would silently over-count.

Step 5 - Why a table beats a CASE expression

The same result could be written as a long `CASE`. The lookup table is better because the bands are **data**: risk can change a threshold with an `UPDATE` and no code release, and every report that joins to `txn_band` changes with it. Interest slabs, tax brackets, charge tiers and reward levels are all handled this way in real banking systems.

Lab 3.4 - Aggregates, GROUP BY and HAVING

Step 1 - Bank-wide numbers

```
SELECT COUNT(*)           AS accounts,
       ROUND(SUM(balance), 2) AS total_deposits,
       ROUND(AVG(balance), 2) AS average_balance,
       ROUND(MIN(balance), 2) AS smallest,
       ROUND(MAX(balance), 2) AS largest
FROM   account;
```

accounts	total_deposits	average_balance	smallest	largest
36	7933404.53	220372.35	4285.90	473345.84

Step 2 - The three COUNTs

```
SELECT COUNT(*)           AS all_rows,
       COUNT(interest_rate) AS rows_with_rate,
       COUNT(DISTINCT account_type) AS distinct_types,
       COUNT(DISTINCT branch_id)   AS branches_represented
FROM   account;
```

COUNT(*) is 36. COUNT(DISTINCT account_type) is 3, because the seed data contains SAVINGS, CURRENT and SALARY but no NRI accounts. **GROUP BY and COUNT DISTINCT only ever show values that exist in the data**, never the full list of values the constraint permits.

Step 3 - Group by branch

```
SELECT a.branch_id,
       COUNT(*)           AS accounts,
       ROUND(SUM(a.balance), 2) AS total_deposits,
       ROUND(AVG(a.balance), 2) AS avg_balance
FROM   account a
WHERE  a.status = 'ACTIVE'
GROUP BY a.branch_id
ORDER BY total_deposits DESC;
```

branch_id	accounts	total_deposits	avg_balance
6	5	1302211.98	260442.40
4	4	1223774.69	305943.67
2	5	1175186.70	235037.34
8	3	1079395.65	359798.55
3	4	961233.96	240308.49
1	4	895617.95	223904.49
5	5	865546.44	173109.29
7	3	262050.93	87350.31

Branch 8 holds the highest average balance on only three accounts, while branch 7 has the lowest. That is a finding the branch head can act on, from one query.

Step 4 - Break the rule on purpose

```
SELECT branch_id, account_number, SUM(balance) FROM account GROUP BY branch_id;
```

```
ERROR: column "account.account_number" must appear in the GROUP BY clause
or be used in an aggregate function
```

Branch 1 has four accounts and the output has one row for branch 1. There is no single correct `account_number` to display, so the engine refuses.

Step 5 - Group by two columns

```
SELECT a.branch_id, a.account_type,
       COUNT(*)           AS accounts,
       ROUND(SUM(a.balance), 2) AS total
FROM   account a
GROUP BY a.branch_id, a.account_type
ORDER BY a.branch_id, a.account_type;
```

Fewer than 24 rows come back, because not every branch holds every account type. `GROUP BY` never invents an empty combination.

Step 6 - WHERE versus HAVING

```
SELECT a.branch_id,
       COUNT(*)           AS accounts,
       ROUND(SUM(a.balance), 2) AS total_deposits
FROM   account a
GROUP BY a.branch_id
HAVING SUM(a.balance) > 1000000
ORDER BY total_deposits DESC;
```

branch_id	accounts	total_deposits
6	5	1302211.98
4	5	1237088.84
2	5	1175186.70
8	3	1079395.65

Four branches hold more than ten lakh. Now try the same condition in `WHERE`:

```
SELECT a.branch_id, SUM(a.balance) FROM account a
WHERE  SUM(a.balance) > 1000000 GROUP BY a.branch_id;
```

```
ERROR: aggregate functions are not allowed in WHERE
```

`WHERE` runs before the rows are grouped, so at that moment no sum exists.

Step 7 - Both together

```
SELECT c.customer_id, c.first_name,  
       COUNT(f.deposit_id)           AS deposits,  
       ROUND(SUM(f.principal_amount), 2) AS total_booked  
FROM   customer c  
       JOIN fixed_deposit f ON f.customer_id = c.customer_id  
WHERE  f.status = 'ACTIVE'  
GROUP BY c.customer_id, c.first_name  
HAVING SUM(f.principal_amount) >= 1000000  
ORDER BY total_booked DESC;
```

customer_id	first_name	deposits	total_booked
1012	Kavya	1	2500000.00
1030	Anjali	1	2500000.00
1010	Priya	1	1500000.00
1028	Pooja	1	1500000.00
1008	Sneha	1	1000000.00
1026	Tara	1	1000000.00

WHERE narrowed the rows to active deposits; **HAVING** then kept only customers whose total reaches ten lakh. Six customers make the wealth team's list.

Lab 3.5 - Pivoting with conditional aggregation

Step 1 - Money in and money out

```
SELECT b.branch_name,
       ROUND(SUM(CASE WHEN t.txn_type = 'CREDIT' THEN t.amount ELSE 0 END), 2) AS money_in,
       ROUND(SUM(CASE WHEN t.txn_type = 'DEBIT' THEN t.amount ELSE 0 END), 2) AS money_out,
       COUNT(*) AS txns
FROM   account_txn t
       JOIN account a ON a.account_id = t.account_id
       JOIN branch b ON b.branch_id = a.branch_id
WHERE  t.status = 'SUCCESS'
GROUP BY b.branch_name
ORDER BY money_in DESC;
```

Two rows of data became two columns. Without `CASE`, answering this would need two queries and a join between them.

Step 2 - Channel mix per branch

```
SELECT b.branch_name,
       COUNT(*) FILTER (WHERE t.channel = 'UPI') AS upi,
       COUNT(*) FILTER (WHERE t.channel = 'ATM') AS atm,
       COUNT(*) FILTER (WHERE t.channel = 'NEFT') AS neft,
       COUNT(*) FILTER (WHERE t.channel = 'BRANCH') AS branch_counter,
       COUNT(*) AS total
FROM   account_txn t
       JOIN account a ON a.account_id = t.account_id
       JOIN branch b ON b.branch_id = a.branch_id
WHERE  t.status = 'SUCCESS'
GROUP BY b.branch_name
ORDER BY total DESC;
```

`FILTER (WHERE ...)` is PostgreSQL's cleaner syntax. The portable equivalent is `SUM(CASE WHEN t.channel = 'UPI' THEN 1 ELSE 0 END)`, which works everywhere.

Step 3 - Channel totals bank-wide

```
SELECT t.channel,
       COUNT(*) AS txns,
       ROUND(SUM(t.amount), 2) AS total_value,
       ROUND(AVG(t.amount), 2) AS avg_ticket
FROM   account_txn t
WHERE  t.status = 'SUCCESS'
GROUP BY t.channel
ORDER BY total_value DESC;
```

channel	txns	total_value	avg_ticket
RTGS	71	4586170.03	64593.94
UPI	64	3922078.62	61282.48
ONLINE	57	3860348.26	67725.41

BRANCH		57		3731729.14		65468.93
POS		56		3407788.62		60853.37
NEFT		61		2687841.41		44062.97
ATM		61		226000.00		3704.92

ATM is the outlier: as many transactions as NEFT, at a twentieth of the average ticket. That is exactly what you would expect - people withdraw small round amounts of cash - and it is the kind of sanity check that tells you the data is behaving.

Step 4 - Subtotals with ROLLUP

```
SELECT COALESCE(b.branch_name, 'ALL BRANCHES') AS branch,
       COALESCE(a.account_type, 'ALL TYPES') AS account_type,
       ROUND(SUM(a.balance), 2) AS total_deposits
FROM   account a
       JOIN branch b ON b.branch_id = a.branch_id
GROUP BY ROLLUP (b.branch_name, a.account_type)
ORDER BY b.branch_name, a.account_type;
```

The final row reads `ALL BRANCHES | ALL TYPES | 7933404.53`, matching the bank-wide total from Lab 3.4. **That agreement is your check** - if the rollup total disagrees with the simple sum, the query has a fan-out problem.

Lab 3.6 - Subqueries

Step 1 - Scalar

```
SELECT account_number, ROUND(balance, 2) AS balance
FROM   account
WHERE  balance > (SELECT AVG(balance) FROM account)
ORDER BY balance DESC;
```

Eighteen of the thirty-six accounts are above the average of 220,372.35.

Step 2 - IN

```
SELECT a.account_number, ROUND(a.balance, 2) AS balance
FROM   account a
WHERE  a.customer_id IN (SELECT customer_id FROM customer WHERE risk_rating = 'HIGH')
ORDER BY a.balance DESC;
```

Step 3 - The NOT IN trap

```
-- Add a deposit row with a NULL nominee already exists; create a NULL-bearing list
SELECT COUNT(*) FROM customer c
WHERE  c.first_name NOT IN (SELECT nominee_name FROM fixed_deposit);
```

```
count
-----
0
```

Zero. Not "few" - none. Six deposits have a NULL `nominee_name`, and one NULL in the list makes every comparison unknown, so no row passes.

Two fixes:

```
-- Exclude the NULLs
SELECT COUNT(*) FROM customer c
WHERE  c.first_name NOT IN (SELECT nominee_name FROM fixed_deposit
                           WHERE nominee_name IS NOT NULL);

-- Or use NOT EXISTS, which is NULL-safe by construction
SELECT COUNT(*) FROM customer c
WHERE  NOT EXISTS (SELECT 1 FROM fixed_deposit f WHERE f.nominee_name = c.first_name);
```

Habit worth forming: reach for `NOT EXISTS` by default. It is correct with NULLs, it never duplicates rows, and it usually plans at least as well.

Step 4 - EXISTS

```
-- Customers holding at least one deposit
SELECT COUNT(*) FROM customer c
WHERE EXISTS (SELECT 1 FROM fixed_deposit f WHERE f.customer_id = c.customer_id);
-- 20

-- Customers holding none
SELECT COUNT(*) FROM customer c
WHERE NOT EXISTS (SELECT 1 FROM fixed_deposit f WHERE f.customer_id = c.customer_id);
-- 10
```

Twenty plus ten is thirty. With `NOT IN` over a NULL-bearing column, the second number would have been zero and the two would not have added up. **Always check that your positive and negative cases sum to the whole.**

Step 5 - Correlated subquery

```
SELECT c.customer_id,
       c.first_name || ' ' || c.last_name AS customer_name,
       (SELECT ROUND(MAX(t.amount), 2)
        FROM account_txn t JOIN account a ON a.account_id = t.account_id
        WHERE a.customer_id = c.customer_id) AS largest_txn
FROM   customer c
ORDER BY largest_txn DESC NULLS LAST
LIMIT 10;
```

Customer 1001 shows a blank, because their only account has never transacted.

Step 6 - Derived table

```
SELECT branch_id, accounts, total_deposits
FROM   (SELECT branch_id,
               COUNT(*)           AS accounts,
               ROUND(SUM(balance), 2) AS total_deposits
        FROM   account
        GROUP BY branch_id) AS branch_summary
WHERE  total_deposits > 1000000
ORDER BY total_deposits DESC;
```

Same four branches as the `HAVING` version in Lab 3.4. Two ways to express one idea. `HAVING` is more direct here; the derived table wins when you need to filter on the aggregate **and** join the summary to something else.

Step 7 - Where a join would be wrong

```
-- Wrong: a customer with 2 accounts is counted twice
SELECT COUNT(*) FROM customer c JOIN account a ON a.customer_id = c.customer_id;
-- 36: the number of accounts, not the number of customers

-- Right
SELECT COUNT(DISTINCT c.customer_id) FROM customer c
       JOIN account a ON a.customer_id = c.customer_id;
-- 30

-- Also right, and clearer about the intent
```

```
SELECT COUNT(*) FROM customer c
WHERE EXISTS (SELECT 1 FROM account a WHERE a.customer_id = c.customer_id);
-- 30
```

Lab 3.7 - Set operators and reconciliation

Step 1 - UNION ALL

```
SELECT customer_id, 'FIXED DEPOSIT' AS product FROM fixed_deposit
UNION ALL
SELECT customer_id, 'LOAN'          AS product FROM loan
ORDER BY customer_id;
```

Thirty-two rows: twenty deposits plus twelve loans.

Step 2 - UNION versus UNION ALL

```
SELECT customer_id FROM fixed_deposit
UNION ALL
SELECT customer_id FROM loan;          -- 32 rows

SELECT customer_id FROM fixed_deposit
UNION
SELECT customer_id FROM loan;          -- fewer, duplicates removed
```

The difference is every customer who holds both products. Decide which number you actually want: "product holdings" is the first, "customers with any product" is the second.

Step 3 - INTERSECT and EXCEPT

```
-- Customers with both a deposit and a loan: the cross-sold base
SELECT customer_id FROM fixed_deposit
INTERSECT
SELECT customer_id FROM loan
ORDER BY customer_id;

-- Deposit holders with no loan: the lending cross-sell list
SELECT customer_id FROM fixed_deposit
EXCEPT
SELECT customer_id FROM loan
ORDER BY customer_id;
```

On Oracle, write `MINUS` instead of `EXCEPT`. Same operator, different name.

Step 4 - Prove the column rules

```
SELECT customer_id, principal_amount FROM fixed_deposit
UNION ALL
SELECT customer_id FROM loan;
```

```
ERROR:  each UNION query must have the same number of columns
```

Step 5 - The reconciliation pattern

This is the pattern you will use to sign off a migration in Module 11. Simulate two extracts:

```
CREATE TABLE core_extract AS
  SELECT txn_id, account_id, amount FROM account_txn WHERE status = 'SUCCESS';

CREATE TABLE warehouse_extract AS
  SELECT txn_id, account_id, amount FROM account_txn WHERE status = 'SUCCESS';

-- Should be empty in both directions
SELECT txn_id, account_id, amount FROM core_extract
EXCEPT
SELECT txn_id, account_id, amount FROM warehouse_extract;

SELECT txn_id, account_id, amount FROM warehouse_extract
EXCEPT
SELECT txn_id, account_id, amount FROM core_extract;
```

Both return zero rows: the systems agree. Now introduce a defect of the kind a real load produces:

```
UPDATE warehouse_extract SET amount = ROUND(amount, 0);

SELECT txn_id, account_id, amount FROM core_extract
EXCEPT
SELECT txn_id, account_id, amount FROM warehouse_extract;
```

Hundreds of rows come back. A rounding difference in the load, invisible in any total that was itself rounded, is caught immediately by a row-level `EXCEPT`.

```
DROP TABLE core_extract;
DROP TABLE warehouse_extract;
```

Lab 3.8 - Assemble the branch performance pack

Ticket ICICI-4530, delivered.

Section 1 - Branch summary

```
SELECT  b.branch_name,
        COUNT(DISTINCT c.customer_id) AS customers,
        COUNT(DISTINCT a.account_id) AS accounts,
        ROUND(SUM(CASE WHEN t.txn_type = 'CREDIT' THEN t.amount ELSE 0 END), 2) AS money_in,
        ROUND(SUM(CASE WHEN t.txn_type = 'DEBIT' THEN t.amount ELSE 0 END), 2) AS money_out
FROM    branch b
        LEFT JOIN customer c ON c.home_branch_id = b.branch_id
        LEFT JOIN account a ON a.customer_id = c.customer_id
        LEFT JOIN account_txn t ON t.account_id = a.account_id
        AND t.status = 'SUCCESS'

GROUP BY b.branch_name
ORDER BY money_in DESC;
```

branch_name	customers	accounts	money_in	money_out
T Nagar Chennai	4	5	2691431.83	1980064.27
Andheri East	4	5	1926049.03	2259434.39
Banjara Hills	4	5	1384840.12	1465615.02
Koramangala	4	4	1334932.58	1580750.78
Connaught Place	3	3	1010101.11	1651285.02
Fort Mumbai Main	4	5	995678.38	2255754.80
Baner Pune	3	4	669734.29	2012828.91
Whitefield	4	5	347121.91	2029982.76

Now remove the two `DISTINCT` keywords and run it again. The customer counts jump into the dozens, because the joins fanned each customer out across their transactions. The totals look plausible and are completely wrong. This is the defect to remember from today.

Section 2 - Dormant customers

```
SELECT  b.branch_name, c.customer_id,
        c.first_name || ' ' || c.last_name AS customer_name, c.phone
FROM    customer c
        JOIN branch b ON b.branch_id = c.home_branch_id
WHERE   NOT EXISTS (
        SELECT 1
        FROM    account_txn t JOIN account a ON a.account_id = t.account_id
        WHERE   a.customer_id = c.customer_id
        AND     t.txn_date >= DATE '2026-01-01')
ORDER BY b.branch_name, c.customer_id;
```

branch_name	customer_id	customer_name	phone
Fort Mumbai Main	1001	Aarav Sharma	9810000000

One customer has not moved a rupee all quarter. The branch has a call to make.

Section 3 - Ten largest transactions

```
SELECT  t.txn_id, b.branch_name,
        c.first_name || ' ' || c.last_name AS customer_name,
        t.txn_date, t.channel, ROUND(t.amount, 2) AS amount
FROM    account_txn t
        JOIN account a ON a.account_id = t.account_id
        JOIN customer c ON c.customer_id = a.customer_id
        JOIN branch b ON b.branch_id = a.branch_id
WHERE   t.status = 'SUCCESS'
ORDER BY t.amount DESC
LIMIT 10;
```

The largest is transaction 700490, an NEFT transfer of 249,247.68 on account 9036.

Section 4 - Deposits maturing soon

```
SELECT  b.branch_name, f.receipt_number,
        c.first_name || ' ' || c.last_name AS customer_name, c.phone,
        f.principal_amount, f.maturity_date, f.auto_renew
FROM    fixed_deposit f
        JOIN customer c ON c.customer_id = f.customer_id
        JOIN branch b ON b.branch_id = f.branch_id
WHERE   f.status = 'ACTIVE'
AND     f.maturity_date < DATE '2026-09-30'
ORDER BY f.maturity_date;
```

Section 5 - Product holdings

```
SELECT  product, COUNT(*) AS holdings, ROUND(SUM(amount), 2) AS total_value
FROM    (SELECT 'FIXED DEPOSIT' AS product, principal_amount AS amount FROM fixed_deposit
        UNION ALL
        SELECT 'LOAN', outstanding_amount FROM loan) AS holdings
GROUP BY product
ORDER BY total_value DESC;
```

End-of-module checklist

	Item
[]	I can write a four-table join and say how many ON conditions it needs
[]	I can explain why a WHERE on the optional table breaks a LEFT JOIN
[]	I can write an anti-join and know to test the primary key for NULL
[]	I used a self join to produce the ICICI Bank branch hierarchy
[]	I built a non-equi join against a band lookup table
[]	I can state the GROUP BY rule and explain why it exists
[]	I know when to use WHERE and when to use HAVING
[]	I pivoted rows into columns with SUM(CASE WHEN ...)
[]	I have seen NOT IN return zero rows because of a NULL
[]	I can explain fan-out and why COUNT(DISTINCT ...) was needed in the pack
[]	I used EXCEPT to reconcile two extracts and caught a rounding defect

Common problems and fixes

Symptom	Cause	Fix
Result has far more rows than expected	Missing join condition	Count them: n tables need n-1
column reference is ambiguous	Same column name in two tables	Alias the tables, qualify the columns
LEFT JOIN returns only matched rows	Condition on the optional table in WHERE	Move it into the ON clause
must appear in the GROUP BY clause	Selected an ungrouped, unaggregated column	Add it to GROUP BY or wrap it in an aggregate
aggregate functions are not allowed in WHERE	Filtering an aggregate	Move the condition to HAVING
NOT IN returns nothing	A NULL in the subquery result	Use NOT EXISTS
Counts far too high in a summary	Fan-out from the join	Use COUNT(DISTINCT ...) , or aggregate before joining
each UNION query must have the same number of columns	Mismatched select lists	Make both sides return the same columns in the same order